



**ΠΑΝΕΠΙΣΤΗΜΙΟ
ΠΑΤΡΩΝ**
UNIVERSITY OF PATRAS

**ΤΜΗΜΑ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ
ΤΕΧΝΟΛΟΓΙΑΣ ΥΠΟΛΟΓΙΣΤΩΝ**

Μάθημα: Προσαρμοστικός Έλεγχος

Τελικό Project

Σάλιαρης Γεώργιος - ΑΜ: 1092584

**Υπεύθυνος καθηγητής: Μπεχλιούλης
Χαραλάμπος**

Ακαδημαϊκό έτος: 2025-26

0. Σκοπός της εργασίας

Έγινε επιλογή του συστήματος του πυράυλου, όπου στόχος είναι η ομαλή προσγείωσή του. Χρησιμοποιούμε 2 είδη ελεγκτών: *adaptive backstepping* (περίπου) και *MIT RULE*. Επίσης έγινε τροποποίηση του περιβάλλοντος προσομοίωσης.

1. Εξισώσεις του συστήματος

$$x = \begin{bmatrix} p_x \\ p_y \\ v_x \\ v_y \\ \theta \\ \omega \\ m \\ u_{act} \\ \delta_{act} \end{bmatrix} \quad (1.1)$$

p_x, p_y : συντεταγμένες του κέντρου μάζας

v_x, v_y : γραμμική ταχύτητα του κέντρου μάζας

θ : γωνιακή εκτροπή του πυραύλου από τον κατακόρυφο άξονα

ω : γωνιακή ταχύτητα του πυραύλου

u_{act}, δ_{act} . Έχουν να κάνουν με την εσωτερική δυναμική του πυραύλου. u_{cmd} με την ώση του πυραύλου, πόσο ώση εμείς θέλουμε για την ακρίβεια. u_{act} είναι αυτό που τελικά κάνει στην πράξη ο κινητήρας αυτήν την στιγμή. δ_{cmd} : η γωνία που δίνουμε εντολή στο ακροφύσιο του πυραύλου να εκτραπεί. δ_{act} : η πραγματική γωνία κλίσης του ακροφυσίου εκείνη την στιγμή. Ισχύουν:

$$\dot{u}_{act} = \frac{u_{cmd} - u_{act}}{\tau_u} \quad (1.2)$$

$$\dot{\delta}_{act} = \frac{\delta_{cmd} - \delta_{act}}{\tau_\delta} \quad (1.3)$$

(τ_u, τ_δ) είναι σταθερές χρόνου που περιγράφουν πόσο γρήγορα προλαβαίνουν να ακολουθήσουν οι κινητήρες τις εντολές

Για λόγους ασφαλείας κάνουμε τα παραπάνω μεγέθη *clamp*:

$$\dot{u}_{act} \leftarrow \text{clamp}(\dot{u}_{act}, u_{min}, u_{max}) \quad (1.4)$$

$$\dot{\delta}_{act} \leftarrow (\dot{\delta}_{act}, \delta_{min}, \delta_{max}) \quad (1.5)$$

Επίσης, προφανώς ισχύει ότι:

$$\dot{p}_x = v_x (1.6)$$

$$\dot{p}_y = v_y (1.7)$$

Για το *thrust* ισχύει ότι:

$$T = u_{act} T_{max} (1.8)$$

$$T_{max} = \frac{T}{W} mg (1.9)$$

Οπότε θα είναι:

$$\frac{T}{m} = \frac{u_{act} 2mg}{m} = 2u_{act}g \left(\frac{T}{W} = 2 \right) (1.10)$$

$$a_y = 2u_{act}g - g = (2u_{act} - 1)g, u_{hover} = 0.5 (1.11)$$

Ειδικά:

$$T_{max} = 2m_0g (1.12), \text{ για } u_{act} = 1$$

Οπότε τελικά θα προκύπτει:

$$u_{hover} = \frac{m}{2m_0} (1.13),$$

δηλ. καίγονται κάυσιμα, μειώνεται η μάζα. οπότε μειώνεται και η u_{hover}

$$\varphi = \theta + \delta_{act} (1.14)$$

$$T_x = T \sin \varphi, T_y = T \cos \varphi (1.15)$$

Επίσης, μοντελοποιούμε και την αντίσταση του αέρα ως εξής:

$$F_{x,drag} = -k ||v_{rel}|| (v_x - w_x), \quad F_{y,drag} = -k ||v_{rel}|| (v_y - w_y) (1.16)$$

$$k = \frac{1}{2} \rho C_d A (1.17)$$

Όπου C_d συντελεστής *drag*, ρ η πυκνότητα του αέρα, A η ενεργός επιφάνεια

$$v_{rel} = \begin{bmatrix} v_x - w_x \\ v_y - w_y \end{bmatrix} (1.18) (w_x, w_y \text{ συνιστώσες της ταχύτητας του ανέμου})$$

$$||v_{rel}|| = \sqrt{(v_x - w_x)^2 + (v_y - w_y)^2} (1.19)$$

$$\rho(p_y) = \rho_0 \exp\left(-\frac{\max(p_y, 0)}{H}\right) \quad (1.20) \quad (H \text{ το αρχικό ύψος})$$

Και από 3^ο NN:

$$\dot{v}_x = \frac{T_x + F_{x,drag}}{m} \quad (1.21)$$

$$\dot{v}_y = \frac{T_y + F_{y,drag}}{m} - g \quad (1.22)$$

Η δε μάζα θεωρούμε ότι μεταβάλλεται σύμφωνα με την παρακάτω εξίσωση:

$$\dot{m} = -\frac{T}{I_{sp}g} \quad (1.23)$$

Όπου στην παραπάνω σχέση I_{sp} είναι μια παράμετρος (η οποία μετριέται σε δευτερόλεπτα) και μας λέει πόσο αποδοτικά χρησιμοποιεί καύσιμο ο κινητήρας. Όσο μεγαλύτερο είναι, τόσο λιγότερο καύσιμο χρειάζεται ο κινητήρας προκειμένου να δώσει το ίδιο *thrust*.

2.Σχεδιασμός Ελεγκτών

A) Adaptive-Backstepping style

Έστω d_x και d_y δύο μεταβλητές οι οποίες κάνουν *estimate* άγνωστες επιταχύνσεις στους δύο άξονες οι οποίες μπορεί να οφείλονται σε

-εξωτερικές διαταραχές (άνεμος πχ)

-Λάθη/αβεβαιότητες στα C_d, A, ρ

- *lag* του κινητήρα (εξισώσεις (1.2) και (1.3))

-θόρυβο που βάζουμε εμείς στα $v_x, v_y, \omega, u_{cmd}, \delta_{cmd}$

κ.α

Σκοπός τους δεν είναι να κάνουν *identify* τον ακριβή λόγο των άγνωστων επιταχύνσεων, αλλά λαμβάνοντας υπόψιν όλους τους παραπάνω παράγοντες να τις ακυρώσουν αυτές τις άγνωστες επιταχύνσεις και έτσι να έχουμε ομαλή προσγείωση του πυραύλου.

Ο τρόπος με τον οποίο γίνονται update τα d_x και d_y είναι ο ακόλουθος:

$$\dot{\hat{d}}_x = \gamma(v_x - v_{x,des})(2.1)$$

$$\dot{\hat{d}}_y = \gamma(v_y - v_{y,des})(2.2)$$

Όπου $v_{x,des}, v_{y,des}$ οι επιθυμητές ταχύτητες στους δύο άξονες. Για να δημιουργηθούν/ενημερωθούν, θα πρέπει να αποφασίσουμε σε ποιο από τα παρακάτω *modes* βρίσκεται ο πύραυλος:

1) **BRAKE**

Στόχος είναι να σταματήσει να πέφτει ο πύραυλος απότομα. Ξεκινάμε από αυτήν την κατάσταση και πηγαίνουμε στην κατάσταση DESCENT όταν έχουν περάσει 0.5 s από την στιγμή που μπήκαμε στην κατάσταση BRAKE και $v_y \geq -11$ (2.3)

Θα είναι:

$$v_{x,des} = 0 \text{ και } v_{y,des} = \max(v_{y,prof}, -8) (2.4)$$

Όπου $v_{y,prof}$ (ισχύει και για τα επόμενα):

```
def _vy_profile(self, y: float) -> float:
    """
    A safe desired descent speed as a function of altitude.

    - high altitude: can descend faster
    - low altitude: must descend slowly

    NOTE: this is conservative by design.
    """
    y = max(0.0, float(y))

    # A simple piecewise profile (easy to understand and tune)
    if y > 120.0:
        return -12.0
    if y > 60.0:
        return -10.0
    if y > 25.0:
        return -7.0

    if y > self.gains.alt_flare:
        return -4.0

    # flare region (y <= alt_flare)
    # at y = alt_flare -> about -2.0
    # at y = 0 -> about -0.3
    return -max(0.3, 0.2 * y)
```

Η λογική του παραπάνω είναι ότι εάν βρισκόμαστε ψηλά, τότε πρέπει ο πύραυλος να κατέβει γρήγορα (αλλά όχι πολύ γρήγορα, εξού και το *max* στην (2.4), καθώς κατεβαίνουμε να μειώνουμε σιγά-σιγά την κατακόρυφη ταχύτητα και κοντά στο έδαφος να πηγαίνουμε πολύ αργά.

2) DESCENT

Εάν δε βρισκόμαστε σε καμία από τις καταστάσεις *FLARE/ALIGN* και ισχύει η συνθήκη εξόδου από την κατάσταση *BRAKE*, εισερχόμαστε στην κατάσταση αυτή. Δηλαδή θα πρέπει

$$10 < p_y$$

$$< 18, \text{βρισκόμαστε εκτός ζώνης προσγείωσης και δεν είμαστε ή μόλις βγήκαμε από } BRAKE \text{ (2.5)}$$

Σε αυτήν την κατάσταση κατεβαίνουμε απλά προς τα κάτω και διορθώνουμε λιγάκι τη συντεταγμένη x , αν χρειαστεί.

$$v_{x,des} = clamp(-0.18 * zone_{error}, -v_{max}, v_{max}) \text{ (2.6)}$$

$$v_{y,des} = v_{y,prof} \text{ (2.7)}$$

Για το δε $zone_error$:

```
# Zone error: 0 inside the zone, distance to nearest edge outside.  
if px < -half:  
    x_zone_error = px + half  
elif px > half:  
    x_zone_error = px - half  
else:  
    x_zone_error = 0.0
```

(Μας λέει πόσο μακριά βρισκόμαστε από τη ζώνη προσγείωσης)

3) *ALIGN*

Στόχος σε αυτήν την κατάσταση είναι να μη προσγειωθεί ο πύραυλος έξω από τη ζώνη. Εισερχόμαστε όταν ισχύει:

$$p_y \leq 18 \text{ και } |p_x| > 5 \text{ (2.8)}$$

Δηλαδή όταν είμαστε χαμηλά και όχι πάνω από την ζώνη προσγείωσης. Κάνουμε hover αργά:

$$v_{y,des} = -0.5 \text{ (2.9)}$$

$$v_{x,des} = clamp(-0.25 * zone_{error}, -v_{max}, v_{max}) \text{ (2.10)}$$

Εδώ έχουμε μεγαλύτερο συντελεστή στο $zone_error$ μιας και θέλουμε μεγαλύτερο sideways correction σε σχέση με το mode *DESCENT*.

4) *FLARE*

(υπερισχύει όλων-δηλαδή, άπαξ και ισχύει η (2.11), εισερχόμαστε σε αυτήν την κατάσταση-ανεξαρτήτως αν ισχύουν κάποια από τις (2.3), (2.5) και (2.8). Σημαίνει ότι είμαστε κοντά στη ζώνη προσγείωσης άρα θα χρειαστεί να πηγαίνουμε αργά:

$$p_y \leq 10 \quad (2.11)$$

$$v_{x,des} = 0 \quad (2.12)$$

$$v_{y,des} = v_{y,prof} \quad (2.13)$$

Ωραία, τώρα έχουμε αποφασίσει σε ποιο *mode* βρισκόμαστε.

Έστω $a_{x,cmd}$ και $a_{y,cmd}$ οι επιθυμητές επιταχύνσεις που θέλουμε να παράγει ο πύραυλος. Τις υπολογίζουμε μέσω ενός *PD* ελεγκτή:

$$\begin{aligned} a_{x,cmd} &= -k_x x_{zone_error} + k_{vx}(v_{x,des} - v_x) \\ a_{y,cmd} &= k_y(y_{ref} - y) + k_{vy}(v_{y,des} - v_y) \end{aligned} \quad (2.14)$$

Επίσης, κάνουμε *clamp* την $a_{y,cmd}$ στο $+g$, καθώς από την (1.13) απορρέει ότι η μέγιστη κατακόρυφη επιτάχυνση είναι ίση με $+g$.

Έστω τώρα a_{thrust} το συνολικό διάνυσμα επιτάχυνσης που πρέπει να έχουμε (προσθέτουμε $+g$ διότι θέλουμε να ακυρώσουμε τη βαρύτητα):

$$a_{thrust} = \begin{bmatrix} a_{x,cmd} \\ a_{y,cmd} + g \end{bmatrix} \quad (2.15)$$

Και αν $u_{act,des}$ και a_{needed} τι θα θέλαμε να παράγει ο κινητήρας και

$$a_{needed} = \sqrt{a_{x,cmd}^2 + (a_{y,cmd} + g)^2} \quad (2.16)$$

Και

$$u_{act,des} = \text{clamp}\left(\frac{a_{needed}}{a_T}, 0, 1\right) \quad (2.17)$$

$$a_T = \frac{T}{W}g = 2g \quad (2.18)$$

$$\theta_{ref} = \phi_{ref} - \delta_{act}$$

(2.19) με ϕ_{ref} να είναι η επιθυμητή γωνία *thrust*

$$\phi_{ref} = \text{atan2}(a_{x,cmd}, a_{y,cmd} + g) \quad (2.20)$$

Αν $\delta_{act,des}$ η επιθυμητή πραγματική γωνία του ακροφυσίου τότε

$$\delta_{act,des} = -\frac{k_{\theta}e_{\theta} + k_{\omega}e_{\omega}}{b_{model}} \quad (2.21)$$

Με $e_{\theta} = \theta - \theta_{ref}$, $e_{\omega} = \omega - 0 = \omega$ (στο $(-\pi, \pi]$) (2.22)

Στην (2.21) $b_{model} =$

$\frac{lT}{I_{yy}}$ (2.23) (I_{yy} η ροπή αδράνειας γύρω από τον άξονα y και l η απόσταση από το κέντρ. μάζας μέχρι εκεί).

εφαρμόζεται το *thrust*

Από την φυσική προκύπτει τελικά ότι:

$\dot{\omega} \approx b_{model}\delta_{act}$ (2.24). Αφού είναι:

$\tau \approx lT\delta_{act}$ (2.25) – πόση ροπή ασκείται στον πύραλο

Και

$$\dot{\omega} \approx \frac{\tau}{I_{yy}} \quad (2.26)$$

Οπότε δεδομένου ότι:

$$\dot{\omega}_{des} = -k_{\theta}e_{\theta} - k_{\omega}e_{\omega} \quad (2.27)$$

προκύπτει η (2.21)

Και εδώ ουσιαστικά έρχεται το *backstepping* κομμάτι

Λόγω της (1.2), ο ελεγκτής που σχεδιάζουμε, επιλέγει το u_{cmd} με τέτοιο τρόπο έτσι ώστε το u_{act} να κάνει *track γρήγορα* το $u_{act,des}$

$$\dot{u}_{act,des} = k_u(u_{act,des} - u_{act}) \quad (2.28)$$

Καταλήγουμε στην εξής εξίσωση:

$$u_{cmd} = u_{act} + \tau_u k_u (u_{act,des} - u_{act}) \quad (2.29)$$

Με όμοιο τρόπο:

$$\delta_{cmd} = \delta_{act} + \tau_{\delta} k_{\delta} (\delta_{act,des} - \delta_{act}) \quad (2.30)$$

Εν τέλει ο ελεγκτής μας θα είναι:

$$u_{cmd,final} = u_{cmd} + 0.01 \widehat{d}_y \quad (2.31)$$

$$\delta_{cmd,final} = \delta_{cmd} - 0.002 \widehat{d}_x \quad (2.32)$$

Η λογική της (2.31) είναι ότι αν $\widehat{d}_y > 0$, αυτό σημαίνει ότι δεν έχουμε αρκετή κατακόρυφη επιτάχυνση, οπότε αυξάνουμε το *throttle* (ειδάλλωξ το μειώνουμε). Η (2.32) μας λέει ότι εάν $\widehat{d}_x > 0$, ο πύραυλος πηγαίνει προς την κατεύθυνση $+x$, οπότε εμείς θέλουμε να τον σπρώξουμε προς την αντίθετη κατεύθυνση (εξού και το ανάποδο πρόσημο). Εάν είναι < 0 , κάνουμε το ανάποδο προφανώς. Τα βάρη και στις δύο εξισώσεις τα ρυθμίζουμε εμπειρικά.

B) MIT RULE

(όπου δεν αναφέρεται ρητά για το πώς βρίσκουμε μια ποσότητα) χρησιμοποιούμε τις εξισώσεις από το προηγούμενο

Ορίζουμε:

$$e_{ay} = a_{y,cmd} - a_{y,mes} \quad (2.33)$$

$$\text{Με } a_{y,mes} = \frac{v_y(t) - v_y(t - \Delta t)}{\Delta t} \quad (2.34)$$

Είναι:

$$a_{T,used} = a_T * \hat{k} \quad (2.35)$$

Στην παραπάνω εξίσωση $\hat{k}$ είναι μία παράμετρος η οποία μας λέει πόσο αποτελεσματικό είναι το thrust που εφαρμόζουμε σε σχέση με το *nominal* μοντέλο. Εμείς ναι μεν μπορεί να κάνουμε *command* κάποιο *throttle*, αλλά ο πύραυλος μπορεί να παράγει περισσότερο/λιγότερο απ'ό,τι το μαθηματικό μοντέλο. Ο τρόπος με τον οποίο γίνεται *update* το $\hat{k}$ είναι ο ακόλουθος:

$$\dot{\hat{k}} = -\gamma_k e_{ay} - \sigma_k (\hat{k} - 1) \quad (2.36)$$

Στη δε παραπάνω εξίσωση, σ_k είναι μία σταθερά η οποία αποτρέπει το $\hat{k}$ να ξεφύγει και το επαναφέρει κοντά στο 1, πράγμα το οποίο επιθυμούμε.

3) ΠΡΟΣΟΜΟΙΩΣΗ

-Απαραίτητες βιβλιοθήκες που πρέπει να γίνουν install (*Windows – pip install*)

pip install numpy, matplotlib, tkinter

Δίνουμε τη δυνατότητα στον χρήστη να προσθέσει θόρυβο στα:

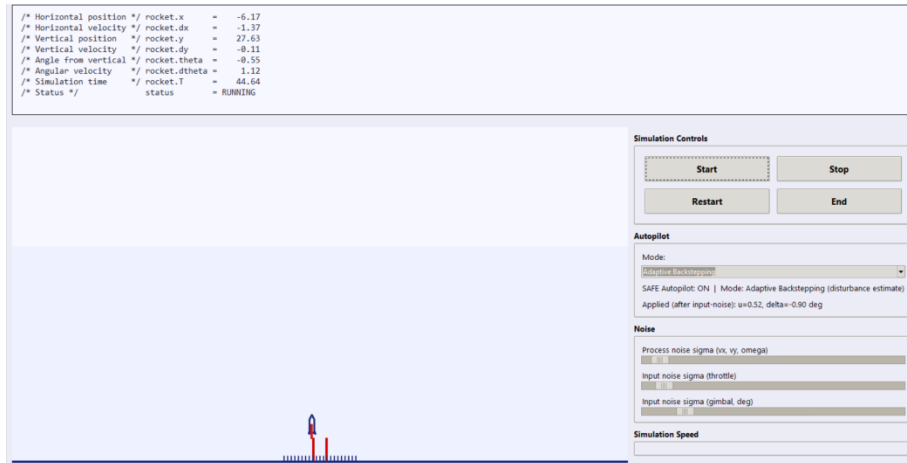
v_x, v_y, ω – *process noise*

u_{used}, δ_{used} – *input noise*

Τρέχοντας το αρχείο *rocket_main.py*, λαμβάνουμε τα εξής αποτελέσματα (autopilot είναι το *control scheme* μας)

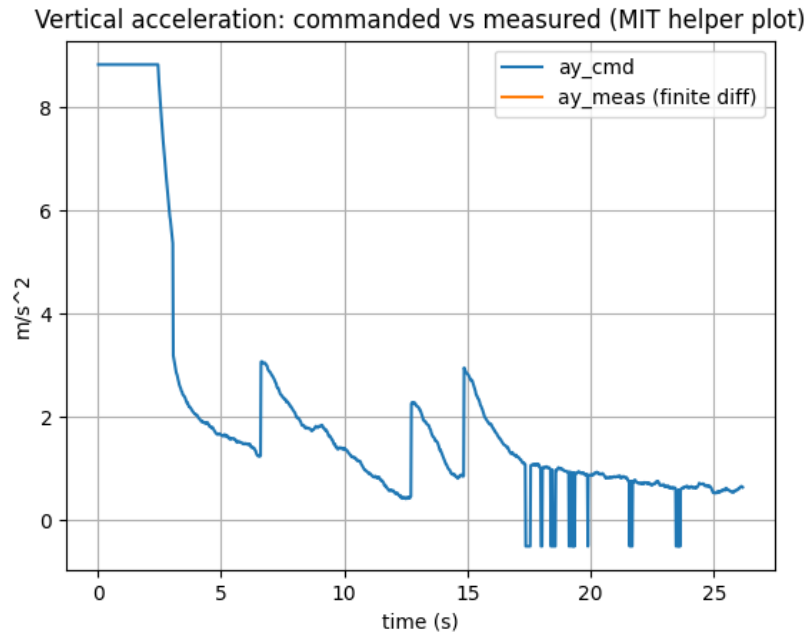
1) Adaptive backstepping:

Όπως φαίνεται και από το *simulation*:

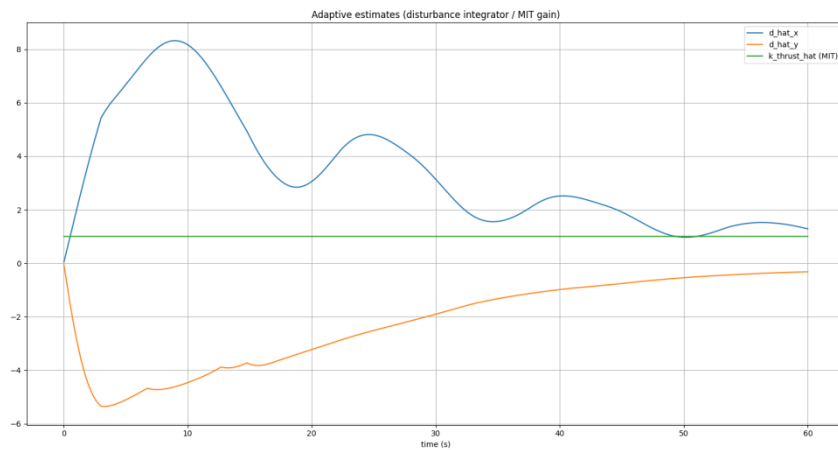


Εικόνα 3.1: Στιγμιότυπο από το simulation από το simulation με *adaptive backstepping*.

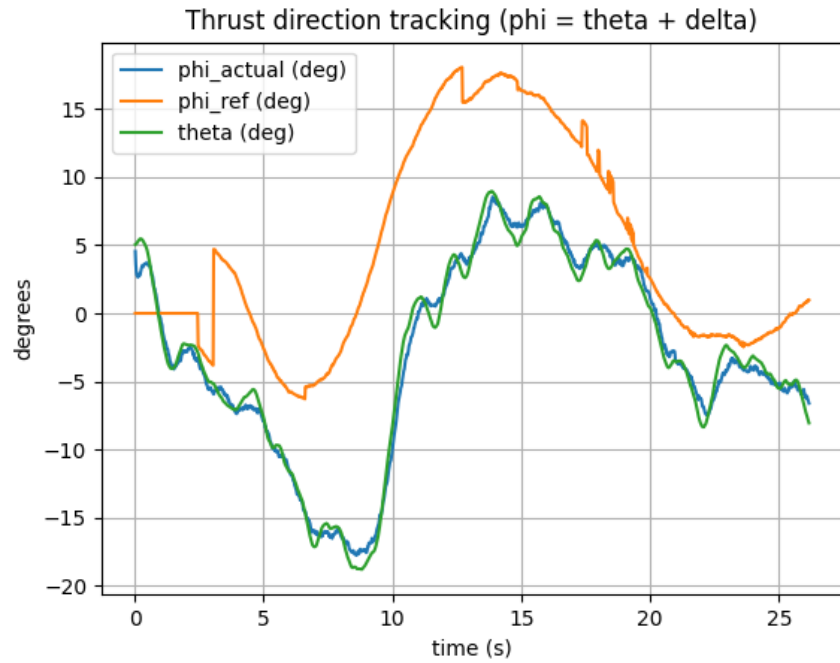
Ο πύραυλος φτάνει πολύ κοντά στην ζώνη προσγείωσης αλλά ταλαντεύεται δεξιά και αριστερά (οπότε ίσως χρειάζεται μεγαλύτερο βάρος στην (2.32). Κατά τα άλλα κατεβαίνει κανονικά κατακόρυφα προς τα κάτω και λαμβάνουμε τα ακόλουθα γραφήματα (για συγκεκριμένες ρυθμίσεις θορύβου):
(Να σημειωθεί ότι επειδή δε επιτυγχάνεται επ' ακριβώς ο στόχος, πατάμε το κουμπί *stop/end* όταν ο πύραυλος βρεθεί σε μία σχετικά καλή θέση).



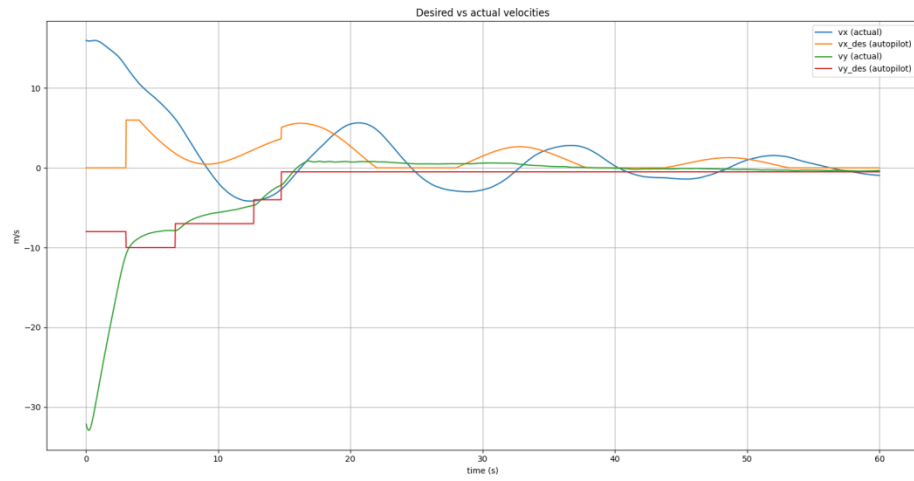
Εικόνα 3.1.1 Κατακόρυφη *commanded* επιτάχυνση. Παρατηρούμε *oscillations* προς τα πάνω, αλλά αυτά δεν είναι τόσο απότομα σε σχέση με τα *oscillations* προς τα κάτω, τα οποία είναι και περισσότερα και πιο απότομα.



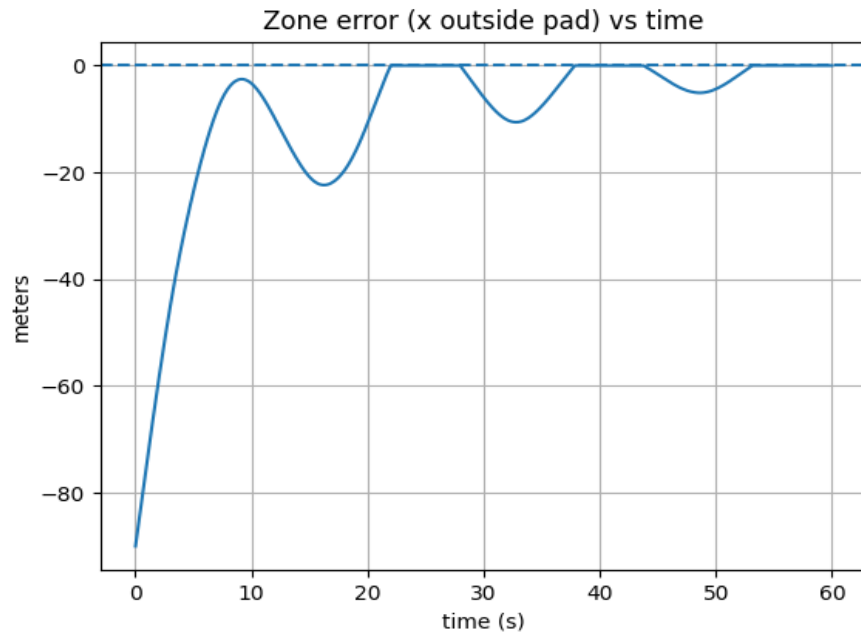
Εικόνα 3.1.2: Εξέλιξη των $\hat{d}_x, \hat{d}_y$ στον χρόνο. Παρατηρούμε ότι τείνουν στο μηδέν, πράγμα λογικό αφού φτάνουμε κοντά στην ισορροπία



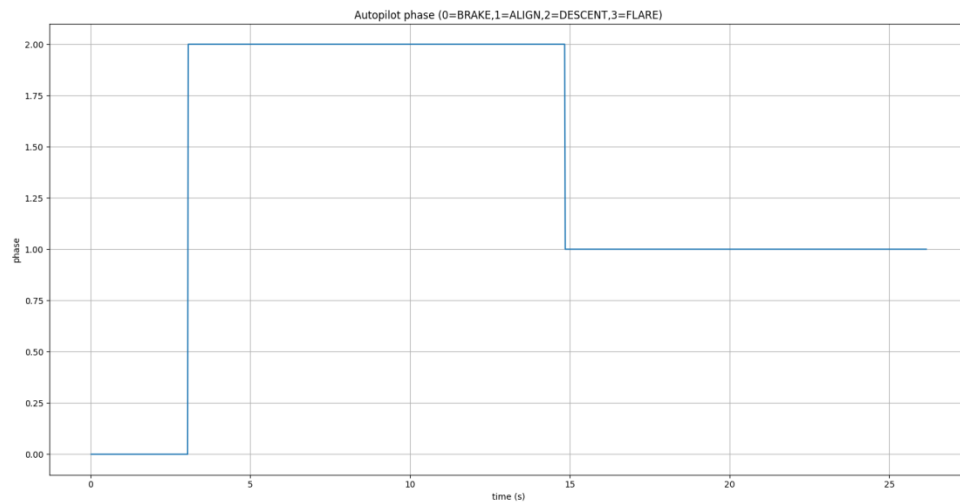
Εικόνες 3.1.3: Γωνίες θ , $\phi_{act} = \theta + \delta_{act}$, ϕ_{ref} . Παρατηρούμε ότι έχουμε απόκλιση στο reference και στην πραγματικότητα αλλά η συμπεριφορά στον χρόνο των δύο (τριών αν σκεφτούμε ότι το θ είναι αρκετά κοντά με το ϕ_{act}).



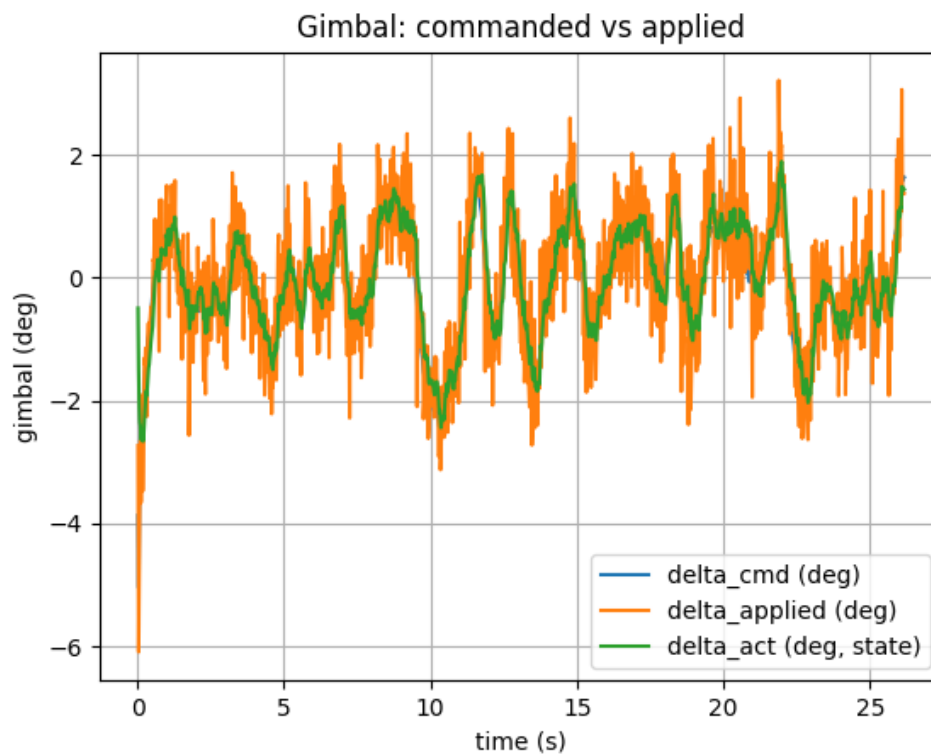
Εικόνα 3.1.4: Εξέλιξη των ταχυτήτων στους δύο άξονες, όπως προκύπτει από τους ελεγκτές και στην πραγματικότητα.



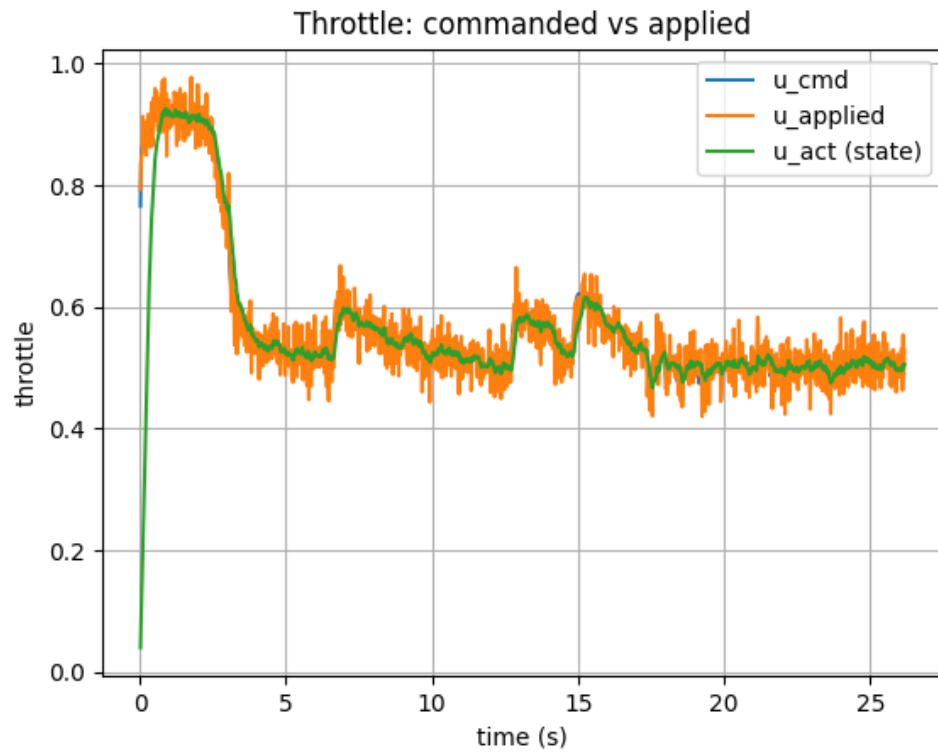
Εικόνα 3.1.5. *Zone-error*-πόσο μακριά βρίσκεται ο πύραυλος από την ζώνη προσγείωσης



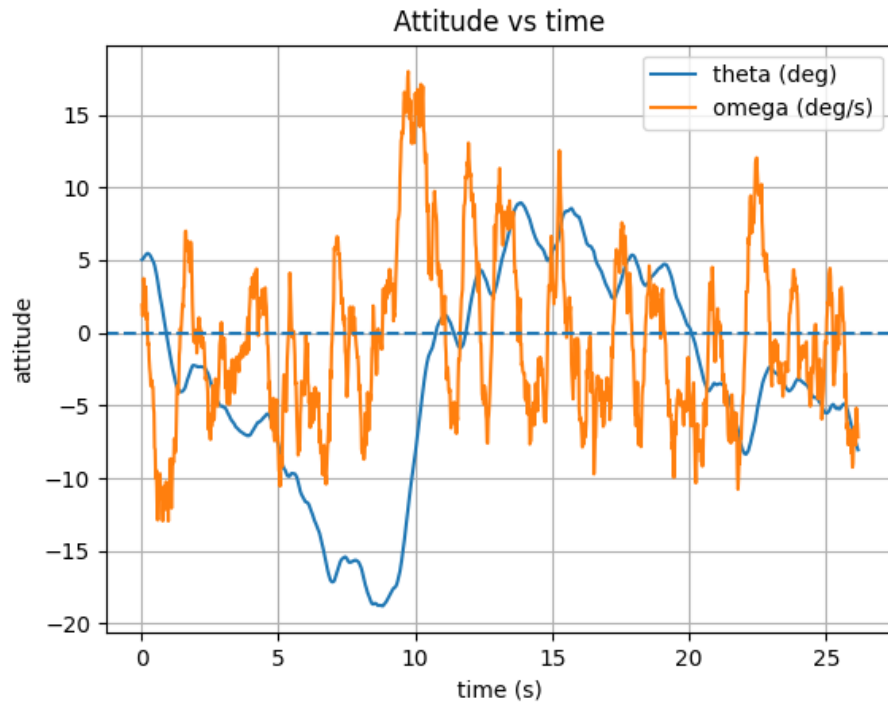
Εικόνα 3.1.6: Modes του πυραύλου. Παρατηρούμε ότι βρίσκεται περίπου στον ίδιο χρόνο στις καταστάσεις *ALIGN* και *DESCENT* ενώ δεν εισέρχεται στην *FLARE*.



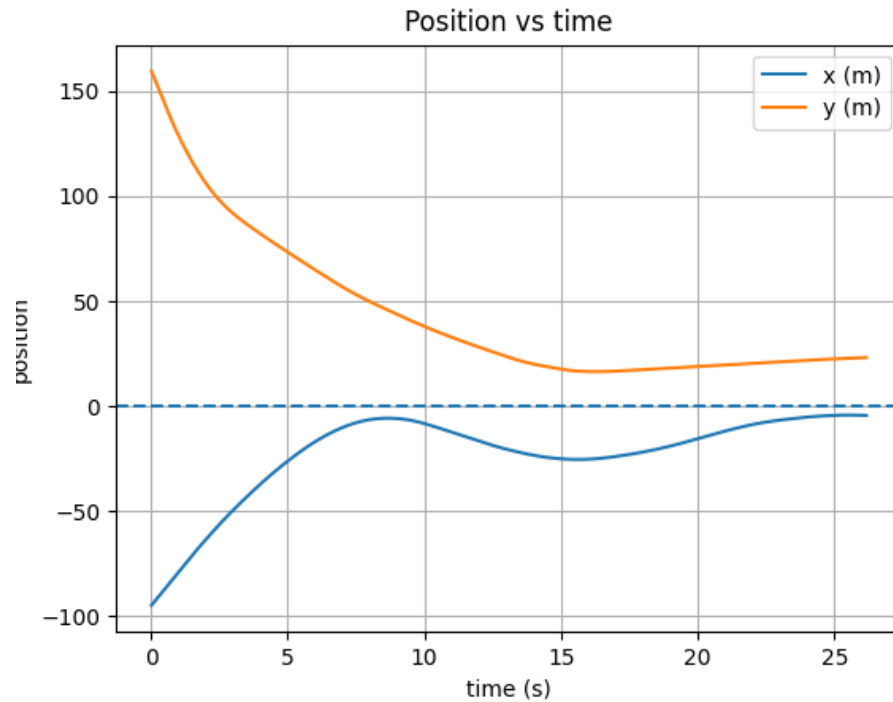
Εικόνα 3.1.7: Εξέλιξη των δ_{cmd} , δ_{act} , δ_{used} . Δεν φαίνεται η δ_{cmd} διότι καλύπτεται από τις άλλες δύο. Πολύ απότομες ταλαντώσεις.

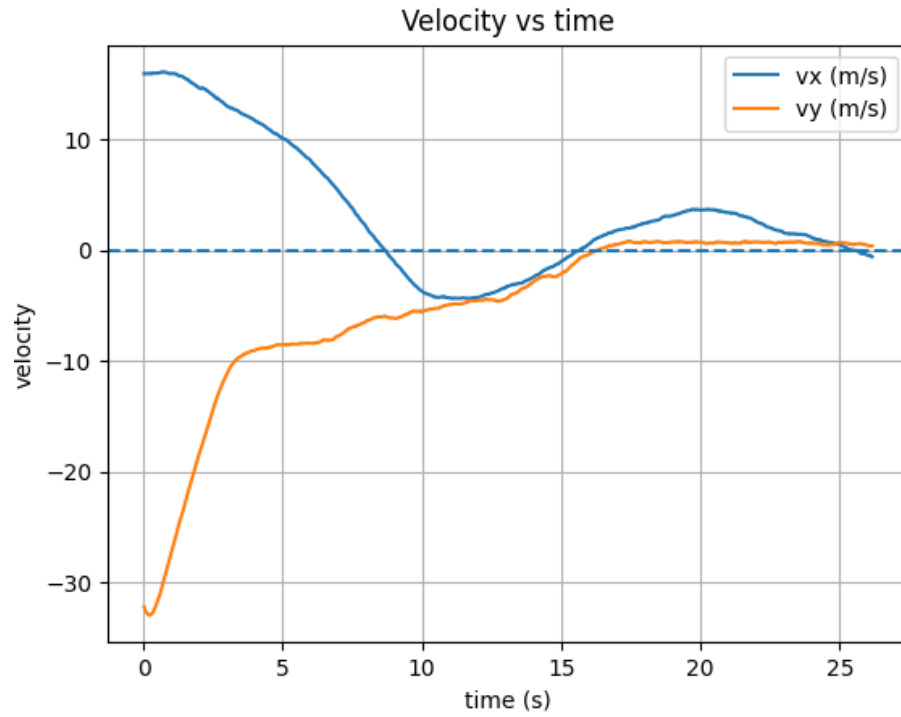


Εικόνα 3.1.8: Εξέλιξη των u_{cmd} , u_{act} , u_{used} . Δεν φαίνεται η u_{cmd} διότι καλύπεται από τις άλλες δύο. Πολύ απότομες ταλαντώσεις.



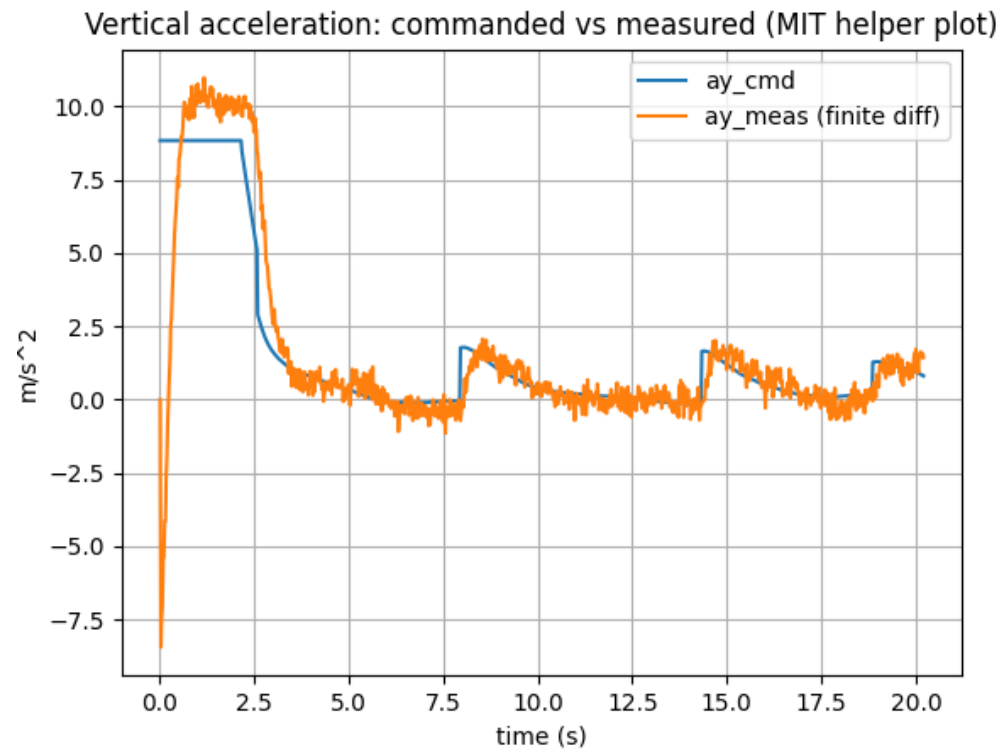
Εικόνα 3.1.9: Εξέλιξη της γωνίας θ και του $\dot{\theta}$ σε σχέση με τον χρόνο. Η γωνιακή ταχύτητα αλλάζει σχετικά απότομα με τον χρόνο, ενώ η θ είναι κατά βάση αρνητική (αντιωρολογιακή φορά).



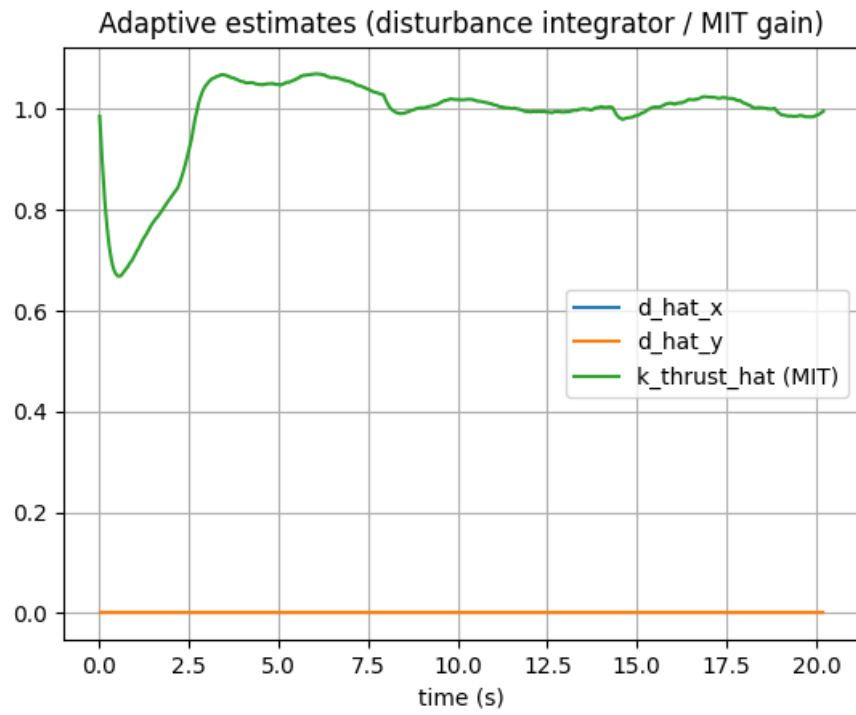


Εικόνα 3.1.10-11: Ταχύτητες και συντεταγμένες του κέντρου μάζας του πυραύλου στον χρόνο. Ω ς αναμενόμενο τείνουν στο 0.

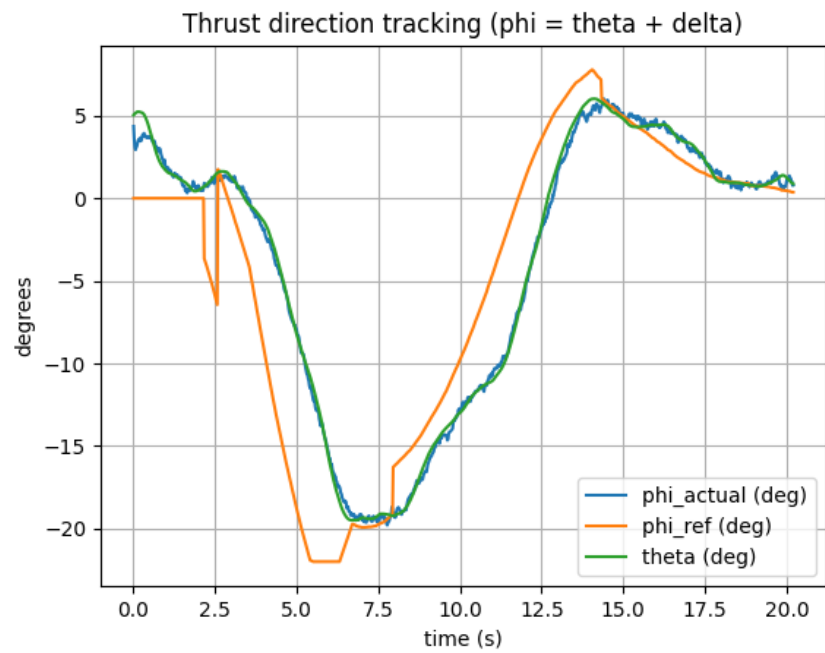
2) MIT



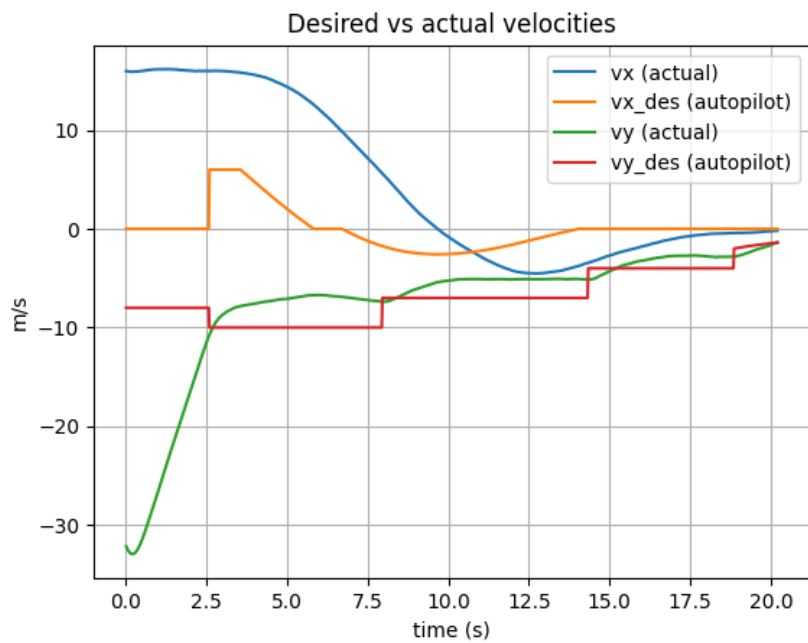
Εικόνα 3.2.1



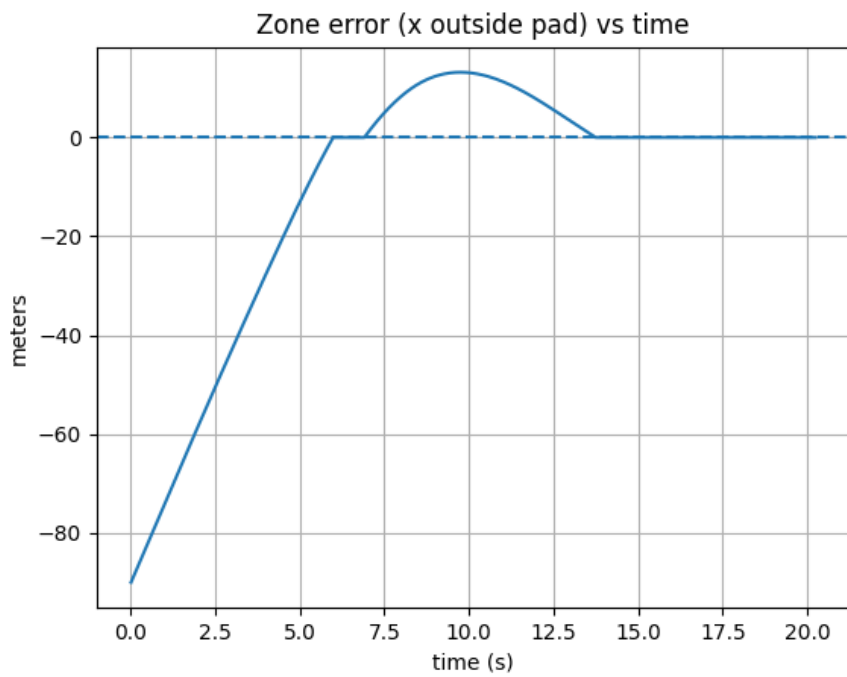
Εικόνα 3.2.2: Εξέλιξη του $\hat{k}$. Τείνει στη μονάδα, το οποίο ακριβώς αυτό θέλουμε.



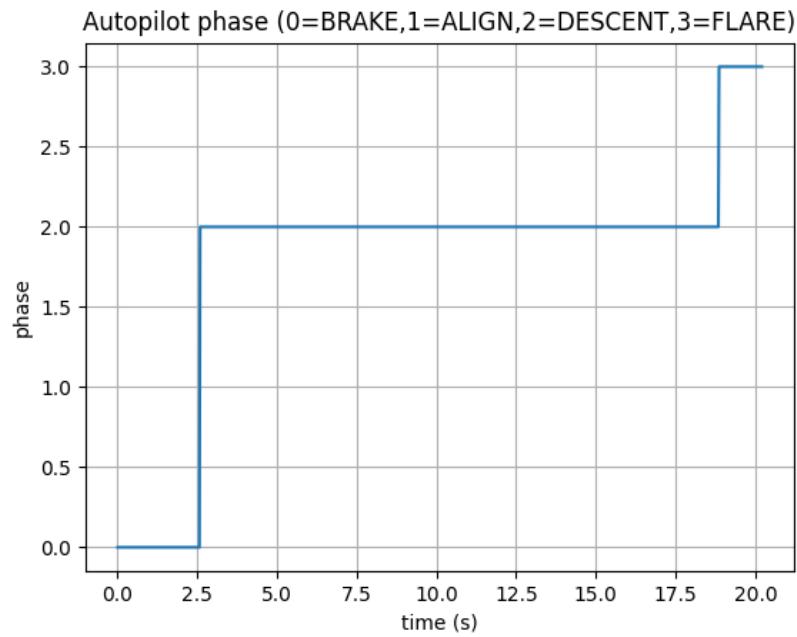
Εικόνα 3.2.3



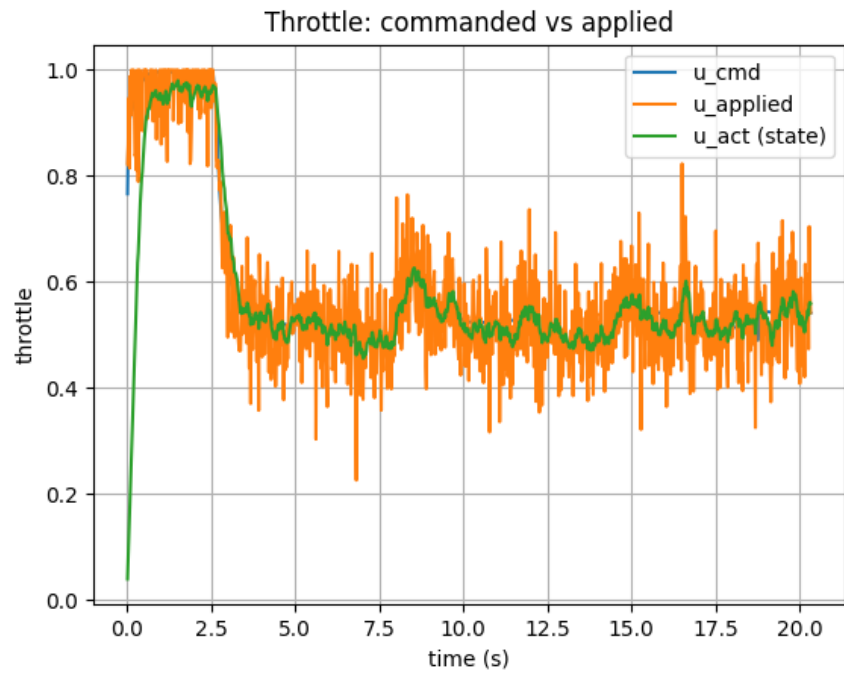
Εικόνα 3.2.4



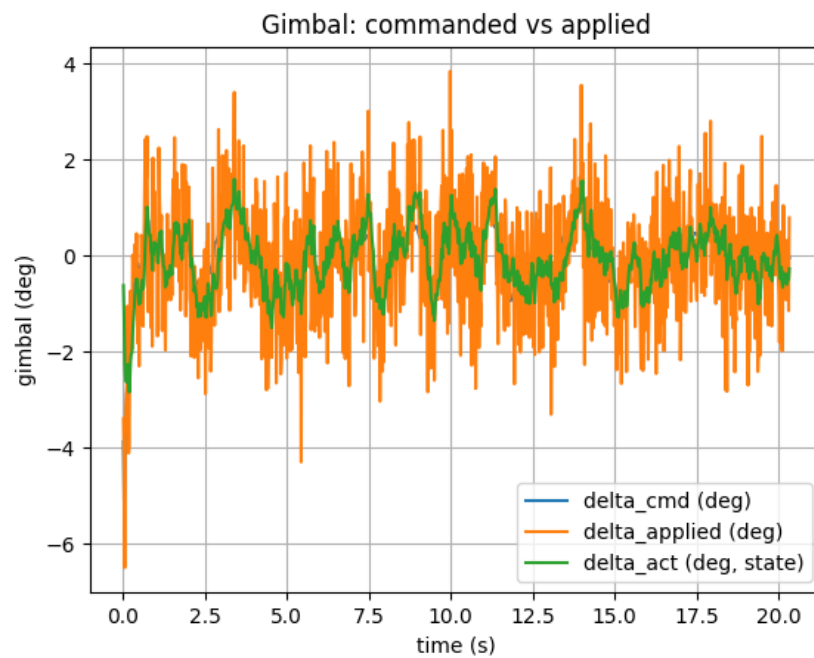
Εικόνα 3.2.5



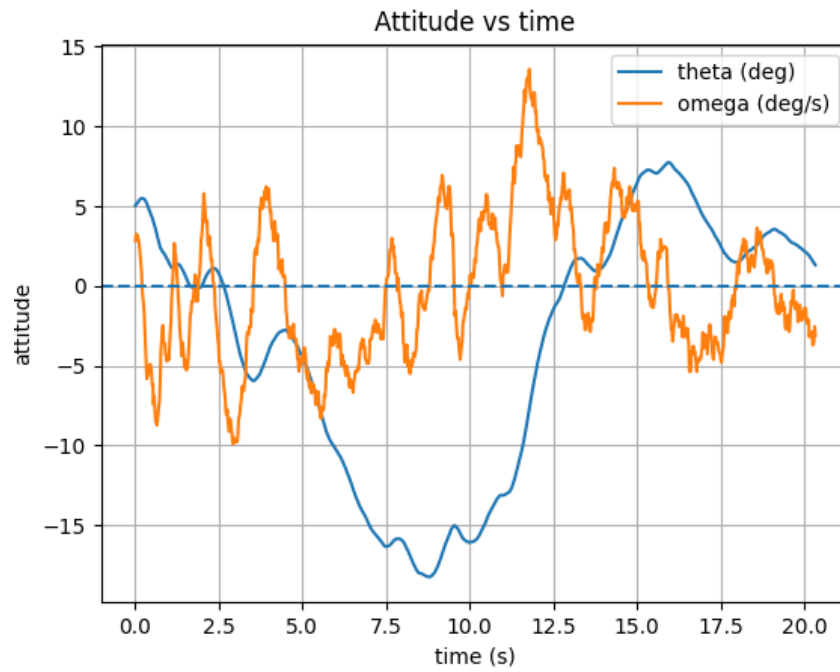
Εικόνα 3.2.6: Εδώ περνάμε από όλες τις καταστάσεις, αλλά είμαστε την περισσότερη ώρα στην κατάσταση *ALIGN*.



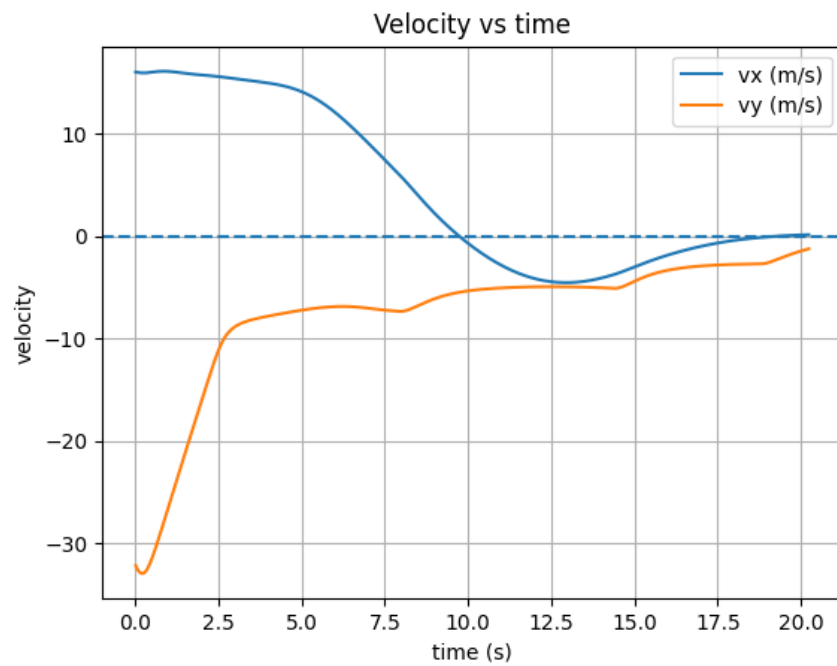
Εικόνα 3.2.7



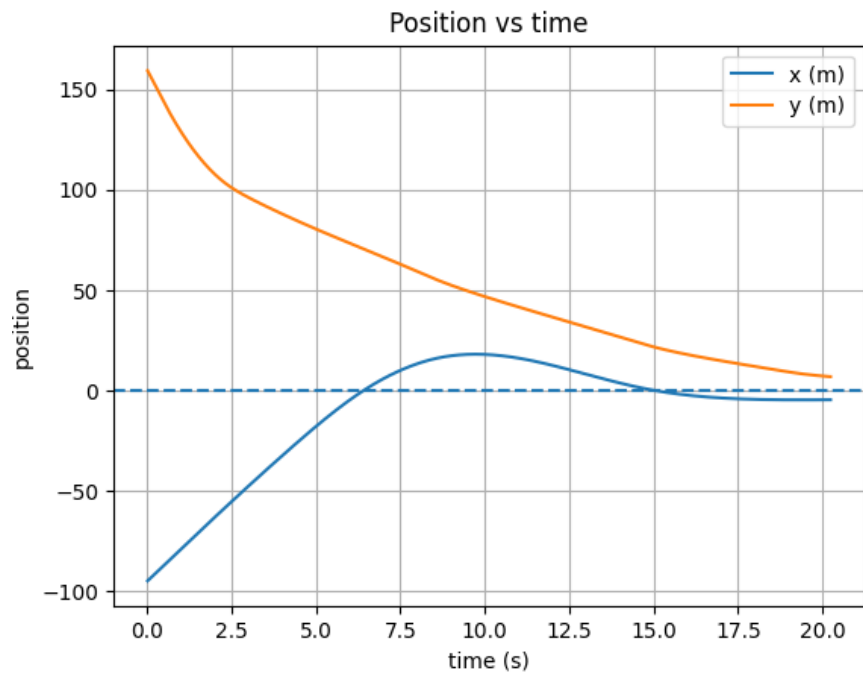
Εικόνα 3.2.8



Εικόνα 3.2.9



Εικόνα 3.2.10



Εικόνα 3.2.11: Το ύψος μειώνεται διαρκώς, ενώ στο x , έχουμε μία μικρή ταλάντωση δεξιά και αριστερά.

Ισχύον παρόμοια σχόλια με την περίπτωση του προηγούμενου ελεγκτή όπου δεν αναφέρεται.

4 ΣΥΜΠΕΡΑΣΜΑΤΑ

Ως γενικό συμπέρασμα, αυτό που μπορούμε να πούμε είναι ότι ο ελεγκτής *MIT* φαίνεται να δουλεύει καλύτερα από τον *adaptive-backstepping* ελεγκτή. Το γεγονός αυτό μπορεί να οφείλεται στην περιπλοκότητα του τελευταίου και στον αριθμό των παραμέτρων που καλείται να ρυθμίσει ο χρήστης. Μάλιστα, παρατηρούμε ότι δεν επιτυγχάνεται με αυτόν τον ελεγκτή το ζητούμενο (να προσγειωθεί ο πύραυλος στην ζώνη) αλλά ταλαντώνεται οριζόντια και εν τέλει η προσομοίωση τελειώνει λόγω μέγιστου χρόνου που έχουμε θέσει εμείς να διαρκεί. Αυτό όμως που μπορούμε να κάνουμε είναι να χρησιμοποιήσουμε τον ελεγκτή αυτόν μέχρι να βρεθούμε κοντά στη ζώνη και μετά να περάσουμε στον *MIT*.

5. Βιβλιογραφία

1. [Guided Tours](#)
2. <https://ntrs.nasa.gov/api/citations/19960003439/downloads/19960003439.pdf>
3. [Adaptive Backstepping Control for Flight Simulator | IEEE Conference Publication | IEEE Xplore](#)
4. [lewisgroup.uta.edu/ee5323/notes/Slotine and Li applied nonlinear control- bad quality.pdf](http://lewisgroup.uta.edu/ee5323/notes/Slotine%20and%20Li%20applied%20nonlinear%20control-%20bad%20quality.pdf)
5. [Robust and Adaptive Control: With Aerospace Applications | Springer Nature Link](#)